

vertcat

Concatenate arrays vertically

Syntax

```
C = vertcat(A,B)
C = vertcat(A1,A2,...,An)
```

Description

`C = vertcat(A,B)` concatenates B vertically to the end of A when A and B have compatible sizes (the lengths of the dimensions match except in the first dimension). [example](#)

`C = vertcat(A1,A2,...,An)` concatenates A1, A2, ..., An vertically. [example](#)

`vertcat` is equivalent to using square brackets to vertically concatenate or append arrays. For example, `[A; B]` is the same as `vertcat(A,B)` when A and B are compatible arrays.

Examples

[collapse all](#)

Two Matrices

Concatenate two matrices vertically.

Create two matrices, and vertically append the second matrix to the first by using square bracket notation.

[Open in MATLAB Online](#)[Copy Command](#)

```
A = [1 2 3; 4 5 6]
```

[Get](#) ▼

A = 2×3

1	2	3
4	5	6

```
B = [7 8 9]
```

[Get](#) ▼

B = 1×3

7	8	9
---	---	---

```
C = [A; B]
```

[Get](#) ▼

C = 3×3

1	2	3
4	5	6
7	8	9

7 8 9

Now, vertically append the second matrix to the first by using `vertcat`.

```
D = vertcat(A,B)
```

[Get](#)

D = 3×3

```
1     2     3
4     5     6
7     8     9
```

Two Tables

Create a table A with three rows and five variables.

[Open in MATLAB Online](#)
[Copy Command](#)

```
A = table([5;6;5],[ 'M'; 'M'; 'M'],[45;41;40],[45;32;34],{'NY';'CA';'MA'},...
    'VariableNames',{'Age' 'Gender' 'Height' 'Weight' 'Birthplace'},...
    'RowNames',{'Thomas' 'Gordon' 'Percy'})
```

[Get](#)

A=3×5 *table*

	Age	Gender	Height	Weight	Birthplace
Thomas	5	M	45	45	{'NY'}
Gordon	6	M	41	32	{'CA'}
Percy	5	M	40	34	{'MA'}

Create a table B with the same variables as A except for order.

```
B = table([ 'F'; 'M'; 'F'],[6;6;5],{'AZ';'NH';'CO'},[31;42;33],[39;43;40],...
    'VariableNames',{'Gender' 'Age' 'Birthplace' 'Weight' 'Height'})
```

[Get](#)

B=3×5 *table*

	Gender	Age	Birthplace	Weight	Height
F	F	6	{'AZ'}	31	39
M	M	6	{'NH'}	42	43
F	F	5	{'CO'}	33	40

Vertically concatenate tables A and B. The variables of C are in the same order as the variables of A and default row names are used for the rows from B.

```
C = vertcat(A,B)
```

[Get](#)

C=6×5 table

	Age	Gender	Height	Weight	Birthplace
Thomas	5	M	45	45	{ 'NY' }
Gordon	6	M	41	32	{ 'CA' }
Percy	5	M	40	34	{ 'MA' }
Row4	6	F	39	31	{ 'AZ' }
Row5	6	M	43	42	{ 'NH' }
Row6	5	F	40	33	{ 'CO' }

Dates with Different Types

Concatenate a date character vector, a string date, and a datetime into a single column of dates. The result is a datetime column vector.

[Open in MATLAB Online](#)
[Copy Command](#)

```
chardate = '2016-03-24';
strdate = "2016-04-19";
t = datetime('2016-05-10','InputFormat','yyyy-MM-dd');
C = vertcat(chardate,strdate,t)
```

[Get](#)

```
C = 3×1 datetime
    24-Mar-2016
    19-Apr-2016
    10-May-2016
```

String Array

Concatenate three string arrays into a single array.

[Open in MATLAB Online](#)
[Copy Command](#)

```
A1 = ["str1" "str2"];
A2 = ["str3" "str4"];
A3 = ["str5" "str6"];
C = vertcat(A1,A2,A3)
```

[Get](#)

```
C = 3×2 string
    "str1"    "str2"
    "str3"    "str4"
    "str5"    "str6"
```

Matrices in a Cell Array

Create a cell array containing two matrices. Vertically concatenate the matrices from the cell array into one matrix.

[Open in MATLAB Online](#)[Copy Command](#)

```
M1 = [1 2; 3 4];  
M2 = [5 6; 7 8];  
A1 = {M1,M2};  
C = vertcat(A1{:})
```

[Get](#) ▼

C = 4x2

1	2
3	4
5	6
7	8

Input Arguments

[collapse all](#)

A — First input

scalar | vector | matrix | multidimensional array | table | timetable

First input, specified as a scalar, vector, matrix, multidimensional array, table, or timetable.

B — Second input

scalar | vector | matrix | multidimensional array | table | timetable

Second input, specified as a scalar, vector, matrix, multidimensional array, table, or timetable.

- The elements of B are concatenated to the end of the first input along the first dimension. The sizes of the input arguments must be compatible. For example, if the first input is a matrix of size 3-by-2, then B must have 2 columns.
- All table inputs must have unique row names. If some of the table inputs do not have row names, `vertcat` fills in default row names for the output. Variable names in each table input must be the same, but the variables can be in a different order. `vertcat` concatenates by matching variable names. For table inputs, `vertcat` also concatenates `RowNames` property values. Other property values of the output table are assigned using the first nonempty (or nondefault) property values of the corresponding inputs.
- Row times of timetable inputs do not need to be unique. Variable names in each timetable input must be the same, but the variables can be in a different order. `vertcat` concatenates by matching variable names. For timetable inputs, `vertcat` also concatenates `RowTimes` property values and events that are attached to the inputs. Other property values of the output timetable are assigned using the first nonempty (or nondefault) property values of the corresponding inputs.

- You can concatenate valid combinations of different types. For more information about concatenating character arrays, logical arrays, and numeric types, see [Valid Combinations of Text and Numeric Datatypes](#).

A1, A2, ..., An — List of inputs

comma-separated list

List of inputs, specified as a comma-separated list of elements to concatenate in the order they are specified.

- The inputs must have compatible sizes. For example, if A1 is a row vector of length m , then the remaining inputs must each have m columns to concatenate vertically.
- All table inputs must have unique row names. If some of the table inputs do not have row names, `vertcat` fills in default row names for the output. Variable names in each table input must be the same, but the variables can be in a different order. `vertcat` concatenates by matching variable names. For table inputs, `vertcat` also concatenates `RowNames` property values. Other property values of the output table are assigned using the first nonempty (or nondefault) property values of the corresponding inputs.
- Row times of timetable inputs do not need to be unique. Variable names in each timetable input must be the same, but the variables can be in a different order. `vertcat` concatenates by matching variable names. For timetable inputs, `vertcat` also concatenates `RowTimes` property values and events that are attached to the inputs. Other property values of the output timetable are assigned using the first nonempty (or nondefault) property values of the corresponding inputs.
- You can concatenate valid combinations of different types. For more information about concatenating character arrays, logical arrays, and numeric types, see [Valid Combinations of Text and Numeric Datatypes](#).

Algorithms

When concatenating an empty array to a nonempty array, `vertcat` omits the empty array in the output. For example, `vertcat([1; 2], [])` returns the column vector `[1; 2]`.

If all input arguments are empty and have compatible sizes, then `vertcat` returns an empty array whose size is equal to the output size as when the inputs are nonempty. For example, `vertcat(zeros(1,0), zeros(2,0))` returns a 3-by-0 empty array. If the input sizes are not compatible, then `vertcat` returns a 0-by-0 empty array.

Extended Capabilities

[collapse all](#)

Tall Arrays

Calculate with arrays that have more rows than fit in memory.

The `vertcat` function supports tall arrays with the following usage notes and limitations:

- Vertical concatenation of character arrays is not supported.
- Concatenation of ordinal categorical arrays is not supported.
- When vertically concatenating two tall arrays, the result is a tall array based on a different datastore than the input tall arrays. Therefore, if you try to combine the result of the concatenation with one of the input tall arrays, the arrays are incompatible. For example, the following command results in an error, since the original tall table `T` is based on one datastore, and the output of the concatenation operation is based on another datastore:

```
T.Var3 = [T.Var1(1); T.Var2(2:end)]
```

Alternatively, if you only concatenate in-memory values to a tall array, then the result of the concatenation is based on the same datastore. For example, if you replace `T.Var1(1)` in the previous example with an in-memory scalar, then the command executes:

```
T.Var3 = [0; T.Var2(2:end)]
```

For more information, see [Tall Arrays](#).

C/C++ Code Generation

Generate C and C++ code using MATLAB® Coder™.

GPU Code Generation

Generate CUDA® code for NVIDIA® GPUs using GPU Coder™.

Thread-Based Environment

Run code in the background using MATLAB® backgroundPool or accelerate code with Parallel Computing Toolbox™ ThreadPool.

The vertcat function fully supports thread-based environments. For more information, see [Run MATLAB Functions in Thread-Based Environment](#).

GPU Arrays

Accelerate code by running on a graphics processing unit (GPU) using Parallel Computing Toolbox™.

The vertcat function supports GPU array input with these usage notes and limitations:

- Sparse matrices are not supported.

For more information, see [Run MATLAB Functions on a GPU](#) (Parallel Computing Toolbox).

Distributed Arrays

Partition large arrays across the combined memory of your cluster using Parallel Computing Toolbox™.

The vertcat function fully supports distributed arrays. For more information, see [Run MATLAB Functions with Distributed Arrays](#) (Parallel Computing Toolbox).

Version History

Introduced before R2006a

See Also

[horzcat](#) | [cat](#)

Topics

[Creating, Concatenating, and Expanding Matrices](#)

[Valid Combinations of Text and Numeric Datatypes](#)

[Combine Categorical Arrays](#)

[Concatenating Objects of Different Classes](#)

[Concatenation Methods](#)

YOU MIGHT ALSO BE INTERESTED IN

Converting Deep Learning Models Between PyTorch, TensorFlow, and MATLAB

Import and export AI models between PyTorch®, TensorFlow™, and MATLAB®.

[Learn more](#)